

KOMA COFFEE

Documentación técnica del proyecto final de grado

Frontend React + Backend Laravel + Docker

Proyecto	Koma Coffee
Tipo	Aplicación web SPA con API REST
Frontend	React + Vite + TypeScript
Backend	Laravel + API REST + MariaDB
Despliegue	Docker Compose + NGINX + Cloudflare Tunnel/dominio

Autor: Jose Luis Sánchez – 2 DAW

IES Pere María d'Orts - Junio de 2026

Nota: esta documentación está orientada a explicar el funcionamiento del proyecto, justificar decisiones técnicas y preparar la defensa. Algunos nombres concretos de rutas o componentes pueden variar ligeramente respecto al código final, pero el flujo técnico es el utilizado durante el desarrollo.

Índice

1. 1. Resumen del proyecto
2. 2. Objetivos y requisitos del TFG
3. 3. Decisiones técnicas principales
4. 4. Arquitectura general
5. 5. Funcionamiento del backend Laravel
6. 6. Base de datos, migraciones y seeders
7. 7. Funcionamiento del frontend React
8. 8. Autenticación, tokens y roles
9. 9. Docker y despliegue
10. 10. Cloudflare Tunnel y dominio público
11. 11. Flujo completo de uso
12. 12. Pruebas recomendadas para la presentación
13. 13. Archivos clave que conviene enseñar
14. 14. Problemas encontrados y soluciones
15. 15. Comandos útiles
16. 16. Preguntas posibles del tribunal
17. 17. Conclusión

1. Resumen del proyecto

Koma Coffee es una aplicación web desarrollada como proyecto final de grado. El objetivo principal es construir una aplicación completa con frontend moderno, backend propio, base de datos, autenticación, roles, CRUDs y despliegue mediante Docker.

El proyecto está planteado como una aplicación de catálogo y gestión. El usuario puede consultar productos/libros/café, iniciar sesión, realizar acciones como reservas y acceder a funcionalidades según su rol. El administrador puede gestionar datos desde un panel protegido.

Área	Tecnología	Función
Frontend	React + Vite + TypeScript	Interfaz de usuario, SPA, rutas, consumo de API.
Backend	Laravel	API REST, validación, autenticación, roles, lógica de negocio.
Base de datos	MariaDB	Almacenamiento de usuarios, categorías, libros/productos y reservas.
Servidor	NGINX	Servir frontend compilado y backend Laravel.
Contenedores	Docker Compose	Levantar todo el entorno de forma reproducible.
Acceso externo	Cloudflare Tunnel / dominio	Permitir abrir la aplicación desde una URL pública.

1.1. Idea funcional

- Catálogo público de productos/libros/café.
- Login y registro de usuarios.
- Diferenciación entre usuario normal y administrador.
- Panel de administración para gestionar entidades.
- Reservas o acciones asociadas a usuarios autenticados.
- Consumo de datos del backend mediante API REST.
- Despliegue local y acceso público mediante túnel/dominio.

2. Objetivos y requisitos del TFG

El proyecto se ha desarrollado teniendo en cuenta los requisitos habituales del módulo y las decisiones tomadas durante el desarrollo. La siguiente tabla resume cómo se cubren los puntos principales.

Requisito	Cómo se cumple en el proyecto	Explicación para defensa
Framework backend	Laravel	Permite crear una API REST estructurada, controladores, modelos, migraciones y autenticación.
Framework frontend	React	Permite crear una SPA modular con componentes reutilizables.

Autenticación por tokens	Laravel Sanctum / tokens API	El backend genera un token al iniciar sesión y React lo envía en peticiones protegidas.
Roles	ADMIN y USER	El campo role del usuario permite diferenciar permisos.
Validación	Controladores/Form Requests	Los datos recibidos en POST/PUT se validan antes de guardarse.
CRUDs	Libros/productos, categorías, usuarios/reservas	Se implementan operaciones GET, POST, PUT/PATCH y DELETE.
Relaciones	Book-Category, User-Reservation, Book-Reservation	Los modelos se conectan mediante claves foráneas.
Eager Loading	with(...) en consultas	Se cargan datos relacionados para evitar consultas innecesarias.
Docker	docker-compose.yml	Levanta backend, frontend, NGINX y base de datos con un solo comando.
Despliegue/dominio	Cloudflare Tunnel	Permite acceder al proyecto mediante URL pública para la presentación.

Funcionalidades especiales como generación de PDFs, envío de emails, jobs o pasarela de pagos deben documentarse como completadas solo cuando estén realmente implementadas. En esta documentación se prioriza lo que ya forma parte del flujo principal de frontend, backend, Docker y demo.

3. Decisiones técnicas principales

3.1. Elección de React para el frontend

React se eligió porque permite crear interfaces por componentes, separar responsabilidades y construir una SPA. En una SPA, la página no se recarga por completo cada vez que se cambia de sección; React actualiza los componentes necesarios.

- Componentes reutilizables como Navbar, tarjetas de producto y páginas.
- Gestión de estado para guardar datos recibidos de la API.
- Rutas internas para navegar entre secciones.
- Integración sencilla con Vite y TypeScript.

3.2. Elección de Laravel para el backend

Laravel se eligió porque ofrece una estructura clara para crear una API REST: rutas, controladores, modelos, migraciones, factories, seeders y autenticación. Además, facilita la validación y el trabajo con relaciones de base de datos mediante Eloquent.

- API REST clara y mantenible.
- Eloquent ORM para trabajar con modelos y relaciones.
- Migraciones para versionar la base de datos.

- Seeders y factories para preparar datos de prueba.
- Autenticación por tokens con Sanctum.

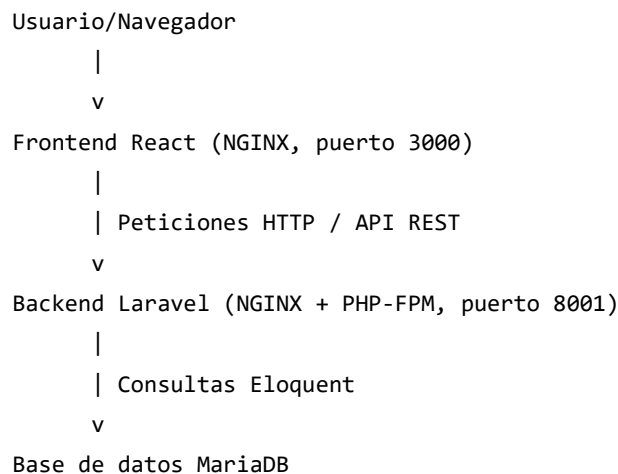
3.3. Elección de Docker

Docker se utilizó para evitar depender de configuraciones manuales en el equipo. Con Docker Compose se levantan todos los servicios necesarios: frontend, backend, base de datos y servidores NGINX.

- Entorno reproducible.
- Menos problemas de versiones de PHP, Node o base de datos.
- Facilita preparar la presentación con comandos concretos.
- Permite simular un despliegue más realista.

4. Arquitectura general

La arquitectura está separada en frontend, backend y base de datos. React no accede directamente a la base de datos. React llama al backend Laravel mediante HTTP, Laravel procesa la petición y consulta MariaDB.



4.1. Separación de responsabilidades

Capa	Responsabilidad
React	Mostrar la interfaz, gestionar estado, navegar entre rutas y consumir la API.
Laravel	Recibir peticiones, validar datos, aplicar reglas de negocio y devolver JSON.
MariaDB	Guardar de forma persistente usuarios, categorías, productos/libros y reservas.
Docker	Levantar y conectar los servicios del proyecto.
NGINX	Servir aplicaciones y redirigir peticiones al servicio correspondiente.

5. Funcionamiento del backend Laravel

El backend funciona como una API REST. Su trabajo es recibir peticiones desde React, operar con la base de datos y devolver respuestas JSON.

5.1. Rutas API: routes/api.php

El archivo routes/api.php define las rutas que puede consumir el frontend. Cada ruta apunta a un método de un controlador.

GET	/api/books	-> listar libros/productos
POST	/api/books	-> crear libro/producto
GET	/api/books/{id}	-> ver detalle
PUT	/api/books/{id}	-> actualizar
DELETE	/api/books/{id}	-> eliminar
POST	/api/login	-> iniciar sesión
POST	/api/logout	-> cerrar sesión
GET	/api/user	-> obtener usuario autenticado

Explicación sencilla: las rutas son las puertas de entrada al backend. React llama a una ruta y Laravel decide qué controlador debe responder.

5.2. Controladores

Los controladores están en app/Http/Controllers. Son clases que contienen la lógica de cada recurso. Por ejemplo, un BookController puede encargarse de listar, crear, editar y borrar productos/libros.

```
public function index()
{
    return Book::with('category')->get();
}
```

En este ejemplo se devuelve una lista de libros junto con su categoría. La función with("category") aplica Eager Loading, es decir, carga la relación de categoría directamente.

5.3. Modelos

Los modelos representan tablas de la base de datos. Por ejemplo, Book representa la tabla books, Category representa categories y User representa users.

```
class Book extends Model
{
    protected $fillable = [
        'title', 'author', 'editorial', 'pages',
        'synopsis', 'image', 'quantity', 'category_id'
    ];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }
}
```

```
}
```

El array fillable indica qué campos se pueden rellenar desde una petición. La relación belongsTo indica que cada libro/producto pertenece a una categoría.

5.4. Validación

Antes de guardar datos, Laravel debe validar la información enviada por el frontend. Esto evita guardar datos incompletos o incorrectos.

```
$request->validate([
    'title' => 'required|string|max:255',
    'category_id' => 'required|exists:categories,id',
    'quantity' => 'required|integer|min:0',
]);
```

Explicación sencilla: si falta un campo obligatorio o tiene un formato incorrecto, Laravel devuelve un error y no guarda el registro.

5.5. Respuestas JSON

Como el frontend y el backend están separados, Laravel devuelve datos en formato JSON. React recibe esos datos y los transforma en elementos visuales.

```
return response()->json([
    'message' => 'Producto creado correctamente',
    'data' => $book
], 201);
```

6. Base de datos, migraciones y seeders

La base de datos guarda la información principal del proyecto. Laravel utiliza migraciones para crear las tablas y seeders/factories para generar datos de prueba.

6.1. Entidades principales

Entidad	Función	Relaciones
User	Guarda usuarios, contraseña y rol.	Puede tener reservas.
Category	Clasifica los productos/libros/café.	Tiene muchos Book.
Book	Elemento del catálogo.	Pertenece a Category y puede aparecer en reservas.
Reservation	Acción de reserva de un usuario.	Pertenece a User y Book.

6.2. Migraciones

Las migraciones son archivos que crean o modifican tablas. Permiten reconstruir la base de datos desde cero de forma controlada.

```
php artisan migrate:fresh --seed
```

Este comando borra las tablas, vuelve a ejecutar las migraciones y carga datos de prueba.

6.3. Tabla users y roles

En la tabla users se guarda el campo role. Este campo permite distinguir entre usuarios normales y administradores.

```
$table->string('role')->default('USER');
```

Rol	Permisos esperados
USER	Puede iniciar sesión, ver catálogo y acceder a funciones de usuario como reservas.
ADMIN	Puede acceder al panel de administración y gestionar datos.

6.4. Tabla reservations

La tabla de reservas conecta usuarios con elementos del catálogo. Tiene claves foráneas a users y books, además de un estado.

```
$table->foreignId('user_id')->constrained()->cascadeOnDelete();
$table->foreignId('book_id')->constrained()->cascadeOnDelete();
$table->string('status')->default('active');
$table->timestamp('reserved_at');
$table->timestamp('expires_at')->nullable();
```

Explicación sencilla: una reserva no existe sola; siempre pertenece a un usuario y a un producto/libro concreto.

6.5. Seeders y factories

Para preparar la presentación se utilizan seeders y factories. Así, la base de datos siempre puede arrancar con datos de prueba.

```
User::factory()->admin()->create([
    'password' => Hash::make('admin123'),
]);
```

```
User::factory()->user()->create([
    'password' => Hash::make('user123'),
]);
```

```
Category::factory(6)->create();
```

```
Book::factory(50)->create();
```

Usuario	Email	Contraseña	Rol
Administrador	admin@koma.com	admin123	ADMIN
Usuario normal	user@koma.com	user123	USER

Explicación para defensa: los seeders permiten reiniciar la base de datos y tener siempre usuarios, categorías y productos preparados para probar la aplicación.

7. Funcionamiento del frontend React

El frontend está desarrollado con React, Vite y TypeScript. React se encarga de mostrar la interfaz, gestionar el estado de la aplicación y comunicarse con Laravel mediante peticiones HTTP.

7.1. SPA

React funciona como una SPA, Single Page Application. Esto significa que la aplicación carga una sola página HTML y después React cambia los componentes visibles según la ruta.

- No se recarga la página completa al navegar.
- La experiencia es más fluida.
- Las rutas muestran componentes diferentes dentro de la misma aplicación.
- El backend solo devuelve datos, normalmente en JSON.

7.2. Punto de entrada: main.tsx

El archivo main.tsx arranca React y monta el componente principal App dentro del HTML.

```
createRoot(document.getElementById('root')!).render(  
  <App />  
);
```

7.3. Rutas: App.tsx

El archivo App.tsx define las rutas principales de la aplicación. Cada ruta carga una página o componente diferente.

```
<Routes>  
  <Route path="/" element={<Home />} />  
  <Route path="/catalog" element={<CoffeeList />} />  
  <Route path="/login" element={<Login />} />  
  <Route path="/admin" element={<ProtectedRoute role="ADMIN"><Admin /></ProtectedRoute>} />  
</Routes>
```

Explicación sencilla: cuando el usuario entra en /catalog, React muestra el catálogo. Cuando entra en /admin, primero se comprueba si tiene permisos.

7.4. Conexión con la API

La conexión con Laravel suele centralizarse en un archivo como services/api.ts o api.ts. Este archivo utiliza la variable de entorno VITE_API_URL para saber dónde está el backend.

```
const API_URL = import.meta.env.VITE_API_URL;  
  
export async function getBooks() {  
  const response = await fetch(`${API_URL}/books`);  
  return response.json();  
}
```

En local, VITE_API_URL apunta a `http://localhost:8001/api`. En despliegue con dominio o Cloudflare debe apuntar a la URL pública del backend.

7.5. Catálogo

Una página como `CoffeeList.tsx` carga los datos del backend y los muestra en pantalla. React guarda esos datos en un estado.

```
const [books, setBooks] = useState([]);

useEffect(() => {
  getBooks().then(data => setBooks(data));
}, []);
```

Explicación sencilla: cuando la página se abre, React pide los datos a Laravel. Cuando llegan, los guarda y actualiza automáticamente la pantalla.

7.6. Componentes reutilizables

React permite dividir la interfaz en componentes. Por ejemplo, `Navbar`, `Footer`, `BookCard` o formularios. Esto evita repetir código y facilita mantenimiento.

- `Navbar`: menú de navegación.
- `BookCard/CoffeeCard`: tarjeta visual de cada elemento del catálogo.
- `Login`: formulario de autenticación.
- `Admin`: zona de gestión protegida.

8. Autenticación, tokens y roles

La autenticación permite identificar al usuario. La autorización permite decidir qué puede hacer ese usuario. En este proyecto se utilizan usuarios con rol `USER` o `ADMIN`.

8.1. Flujo de login

1. El usuario introduce email y contraseña en React.
2. React envía una petición POST a `/api/login`.
3. Laravel valida las credenciales.
4. Si son correctas, Laravel devuelve un token y los datos del usuario.
5. React guarda el token.
6. En peticiones protegidas, React envía el token en la cabecera `Authorization`.

8.2. Token

El token funciona como una identificación temporal. Permite que Laravel sepa qué usuario está haciendo la petición sin tener que enviar email y contraseña constantemente.

```
Authorization: Bearer TOKEN_DEL_USUARIO
```

8.3. Control de roles

El campo `role` permite controlar permisos. Por ejemplo, un usuario `ADMIN` puede entrar al panel de administración, mientras que un usuario `USER` no debería poder hacerlo.

```
if (user.role !== 'ADMIN') {
```

```
    return <Navigate to="/" />;  
  }  
}
```

Punto importante: si en la base de datos los roles están en mayúsculas, el frontend y backend deben comparar exactamente ADMIN y USER, no admin y user.

9. Docker y despliegue local

Docker permite levantar todo el proyecto con una configuración controlada. El archivo principal es docker-compose.yml.

9.1. Contenedores del proyecto

Contenedor	Función	Puerto
koma_frontend_react	Sirve React compilado mediante NGINX.	3000 -> 80
koma_nginx_laravel	Servidor NGINX para Laravel.	8001 -> 80
koma_app_laravel	Aplicación Laravel/PHP-FPM.	9000 interno
koma_db	Base de datos MariaDB.	3306 interno/según configuración

9.2. Comandos para levantar el proyecto

```
docker compose down -v  
docker compose up -d --build  
docker ps
```

Explicación: primero se bajan los contenedores, después se reconstruye y levanta todo, y finalmente se comprueba que los servicios están activos.

9.3. Preparar Laravel después de levantar Docker

```
docker exec -it koma_app_laravel bash  
php artisan key:generate  
php artisan optimize:clear  
php artisan migrate:fresh --seed  
exit
```

Con estos comandos se genera la clave de Laravel si hace falta, se limpian cachés y se prepara la base de datos con datos de prueba.

9.4. URLs locales importantes

Servicio	URL
Frontend React	http://localhost:3000
Backend Laravel	http://localhost:8001
API catálogo	http://localhost:8001/api/books

10. Cloudflare Tunnel y dominio público

Para enseñar la aplicación desde fuera del ordenador local se puede usar Cloudflare Tunnel. Es importante exponer tanto frontend como backend, porque React necesita llamar a la API.

10.1. Túnel para backend

```
cloudflared tunnel --url http://localhost:8001
```

Este comando genera una URL pública para la API de Laravel. Esa URL debe usarse en VITE_API_URL.

10.2. Túnel para frontend

```
cloudflared tunnel --url http://localhost:3000
```

Este comando genera una URL pública para abrir la aplicación React desde cualquier dispositivo.

10.3. Variable VITE_API_URL

El frontend compilado necesita saber dónde está la API. Para demo local puede ser localhost. Para dominio público debe ser la URL pública del backend.

```
# Demo local
```

```
VITE_API_URL=http://localhost:8001/api
```

```
# Demo con dominio o Cloudflare
```

```
VITE_API_URL=https://url-publica-backend.trycloudflare.com/api
```

Muy importante: si se cambia .env.production, hay que reconstruir el frontend porque Vite inserta esa variable durante el build.

```
docker compose build frontend
```

```
docker compose up -d frontend
```

10.4. CORS

Si frontend y backend están en dominios distintos, Laravel debe permitir el origen del frontend en config/cors.php.

```
allowed_origins => [  
    'http://localhost:3000',  
    'http://127.0.0.1:3000',  
    'https://url-publica-frontend.trycloudflare.com',  
]
```

Después de cambiar CORS conviene limpiar caché de Laravel.

```
docker exec -it koma_app_laravel bash
```

```
php artisan optimize:clear
```

```
exit
```

11. Flujo completo de uso

11.1. Cargar catálogo

1. El usuario abre React.
2. React carga la página del catálogo.
3. React hace GET a /api/books.
4. Laravel recibe la ruta en routes/api.php.
5. El controlador consulta Book::with('category').
6. Laravel devuelve JSON.
7. React pinta tarjetas en pantalla.

11.2. Crear un elemento desde admin

1. El admin inicia sesión.
2. React guarda el token.
3. El admin rellena el formulario.
4. React envía POST /api/books con el token.
5. Laravel valida los datos.
6. Laravel guarda el registro.
7. React actualiza la lista.

11.3. Hacer una reserva

1. El usuario inicia sesión.
2. Selecciona un elemento del catálogo.
3. React envía la petición de reserva.
4. Laravel asocia user_id y book_id.
5. Se crea una reserva con estado active.
6. El usuario puede verla en su zona de reservas.

12. Pruebas recomendadas para la presentación

Para evitar improvisar, conviene hacer siempre las pruebas en el mismo orden.

Orden	Prueba	Resultado esperado
1	docker ps	Todos los contenedores están activos.
2	Abrir http://localhost:8001/api/books	Se ve JSON del backend.
3	Abrir http://localhost:3000	Carga la aplicación React.
4	Catálogo	Se muestran datos procedentes de Laravel.
5	Login USER	user@koma.com entra correctamente.
6	Zona usuario/reservas	El usuario puede ver sus acciones.
7	Intentar admin con USER	No debe permitir acceso.

8	Login ADMIN	admin@koma.com entra correctamente.
9	Panel admin	Puede crear, editar y eliminar datos.
10	Dominio/Cloudflare	La aplicación carga desde URL pública y llama a API pública.

12.1. Orden recomendado para la demo de 15 minutos

- Presentar brevemente la idea del proyecto.
- Enseñar docker ps para demostrar los contenedores.
- Abrir /api/books para enseñar el backend JSON.
- Abrir el frontend y explicar que React es SPA.
- Mostrar el catálogo.
- Iniciar sesión como usuario normal.
- Mostrar una funcionalidad de usuario, como reservas.
- Cerrar sesión e iniciar sesión como administrador.
- Mostrar panel admin y un CRUD.
- Cerrar explicando Docker, dominio y posibles mejoras.

13. Archivos clave que conviene enseñar

Archivo	Qué explicar
docker-compose.yml	Define los servicios: frontend, backend, NGINX y base de datos.
backend/routes/api.php	Rutas REST que consume React.
backend/app/Http/Controllers/BookController.php	Lógica para listar, crear, editar y borrar datos.
backend/app/Models/Book.php	Modelo principal y relación con Category.
backend/app/Models/User.php	Usuario, tokens y campo role.
backend/database/migrations	Estructura de tablas y claves foráneas.
backend/database/seeder/DatabaseSeeder.php	Datos preparados para la demo.
backend/config/cors.php	Permisos para que React pueda llamar a Laravel.
frontend/src/App.tsx	Rutas de la SPA.
frontend/src/main.tsx	Arranque de React.
frontend/src/services/api.ts	Conexión centralizada con la API.
frontend/src/pages/CoffeeList.tsx	Consumo de API y pintado del catálogo.
frontend/src/pages/Login.tsx	Envío de credenciales y recepción de token.
frontend/src/context/AuthContext.tsx	Gestión global de sesión.
frontend/src/components/ProtectedRoute.tsx	Protección de rutas por login y rol.
frontend/.env.production	URL del backend para producción o dominio.

14. Problemas encontrados y soluciones

Problema	Causa	Solución aplicada
El catálogo no cargaba datos	CORS o URL de API incorrecta	Configurar allowed_origins y VITE_API_URL.
Puerto ocupado	El puerto 8000 estaba ocupado	Cambiar backend a 8001.
Docker Compose daba error en networks	Propiedades mal colocadas en docker-compose.yml	Reorganizar servicios, redes y volúmenes correctamente.
Frontend NGINX devolvía Empty reply	Configuración NGINX incorrecta	Corregir default.conf y reconstruir contenedor.
Factories fallaban	Uso incorrecto de fake/faker	Ajustar factories y seeder.
Logo o imágenes no cargaban	Ruta incorrecta en React	Usar public y rutas como /logo.png o /menu.png.
Cloudflare cargaba web pero no API	El frontend seguía apuntando a localhost	Cambiar VITE_API_URL a la URL pública del backend y reconstruir frontend.

15. Comandos útiles

15.1. Docker

```
docker compose down -v
docker compose up -d --build
docker ps
docker logs koma_app_laravel
docker logs koma_nginx_laravel
docker logs koma_frontend_react
```

15.2. Laravel dentro del contenedor

```
docker exec -it koma_app_laravel bash
php artisan key:generate
php artisan optimize:clear
php artisan migrate:fresh --seed
php artisan route:list
exit
```

15.3. Comprobaciones rápidas

```
curl http://localhost:8001/api/books
curl -I -H "Origin: http://localhost:3000" http://localhost:8001/api/books
```

15.4. Cloudflare Tunnel

```
cloudflared tunnel --url http://localhost:8001
```

```
cloudflared tunnel --url http://localhost:3000
```

16. Preguntas posibles del tribunal

Pregunta	Respuesta recomendada
¿Por qué React es una SPA?	Porque carga una única página HTML y React cambia los componentes según la ruta sin recargar todo el documento.
¿Por qué Laravel?	Porque facilita crear una API REST estructurada con rutas, controladores, modelos, validación, migraciones y autenticación.
¿Qué es una API REST?	Un conjunto de rutas HTTP que permiten obtener, crear, modificar y borrar recursos usando JSON.
¿Cómo funciona el login?	React envía credenciales a Laravel; Laravel valida y devuelve un token; React usa ese token en peticiones protegidas.
¿Cómo controlas los roles?	El usuario tiene un campo role con ADMIN o USER. Ese rol se comprueba para permitir o bloquear rutas.
¿Qué es Eager Loading?	Es cargar relaciones desde el principio, por ejemplo Book con Category, para evitar consultas adicionales innecesarias.
¿Por qué Docker?	Para levantar todo el entorno de forma reproducible sin configurar manualmente PHP, Node, NGINX y MariaDB.
¿Qué problema resuelve CORS?	Permite que un frontend servido desde un dominio o puerto pueda llamar a un backend diferente.
¿Por qué con Cloudflare no sirve localhost?	Porque desde otro dispositivo localhost apunta a ese dispositivo, no a mi ordenador; por eso necesito una URL pública para la API.
¿Qué mejorarías?	Mejorar diseño, completar funcionalidades especiales, añadir más tests, reforzar seguridad y preparar despliegue permanente.

17. Conclusión

Koma Coffee demuestra una arquitectura web moderna separando frontend y backend. React se encarga de la interfaz y la experiencia de usuario, mientras Laravel gestiona la API, la base de datos, autenticación y reglas de negocio. Docker permite ejecutar todo el proyecto con una configuración reproducible y Cloudflare Tunnel o un dominio permiten acceder a la aplicación desde una URL pública.

Las decisiones técnicas tomadas se justifican por la claridad, facilidad de mantenimiento y adecuación a los requisitos del proyecto final. La separación por capas permite explicar bien el flujo completo: el usuario interactúa con React, React consume Laravel, Laravel consulta MariaDB y devuelve datos JSON que React muestra en pantalla.

